BaiZe: From-Scratch Vision Encoder and Multimodal Grounding for Visual EDA Tasks

Abstract

Agentic LLMs for electronic design automation (EDA) are already deployed in-house, with models such as DeepSeek-V4-Flash and GLM-5.3-Flash serving as the coordinating master. Cost, not feasibility, is now the binding constraint: every EDA subtask routed to a frontier master consumes tokens and serving budget that scale with design-flow volume. We present **BaiZe**, a $\approx$2.2B hybrid state-space agentic model built from scratch as a cost-efficient EDA subagent beneath such a master: it receives subtasks in natural language and drives EDA tools in their native scripting languages. Its 56 layers interleave Mamba-2 state-space and MLP layers with four attention layers, so only the latter carry a session-growing state; decoding is linear-time and per-subtask serving cost is governed by session memory rather than prompt throughput. A matched from-scratch comparison against a dense transformer selects this architecture (11.6% fewer parameters, lower loss at a matched 1k-step budget, and $+10.5\%$ prefill throughput), and a budget-constrained search fixes the pre-training recipe, now scaled to a 2.94T-token production run. The backbone now stands at training loss 2.151 after 30,660 iterations ($\approx$598K tok/s), and the encoder reaches 19.76% / 20.27% ImageNet linear probe at 55.3M / 176M samples; a larger ViT-L/14 tower reaches 68.2% on an internal protocol, yet the MLLM stage averages 26.1% over 11 multimodal benchmarks, showing that encoder gains do not transfer downstream at the 224-px single-tile setting. On a 449-task EDA-scripting benchmark spanning three languages and four commercial runtimes **?**, we test whether BaiZe can offload subtasks from the deployed Flash masters at comparable task accuracy and materially lower serving cost, and whether a from-scratch vision encoder's 196-token-per-image budget suffices for detail-dense layout, schematic, and waveform views. Later stages are specified as protocols with pre-specified criteria; their measurements are in progress, and we release the full recipe and evaluation protocol.

small language model, agentic LLM, hybrid state-space architecture, EDA subagent, cost-efficient inference, multimodal grounding

1 Introduction

Large language models have recently demonstrated agentic capabilities—planning, tool invocation, and long-horizon reasoning—that make them attractive cores for intelligent assistants in electronic design automation (EDA). Such assistants are no longer hypothetical: agentic EDA flows are already deployed in-house, with Flash-tier masters such as DeepSeek-V4-Flash and GLM-5.3-Flash coordinating the flow. The binding constraint is therefore cost, dominated by two terms: every subtask routed to a master consumes tokens and serving budget that scale with flow volume, and because an agentic session accumulates context, serving cost is driven by the memory that grows with the session rather than by prompt throughput. A small, locally served subagent caps the cost of every subtask it takes over by pricing it at a local 2.2B model rather than at a Flash master. Building such a model *from scratch* as a production subagent, however, remains an under-documented engineering problem.

Contributions. We present BaiZe, a ≈2.2B-parameter hybrid (state-space + attention) agentic model built from scratch to serve as a cost-efficient EDA subagent beneath a deployed Flash master.

- **A cost-efficient EDA subagent.** BaiZe receives EDA subtasks in natural language from the master and drives EDA tools in their native scripting languages (PyAether, SKILL, Tcl). Its 56 layers interleave Mamba-2 state-space and MLP layers with four grouped-query-attention layers, so only the latter carry a session-growing state—which is what makes per-subtask cost reduction structural rather than a matter of quantization alone. A staged budget-constrained search (S1–S5) with a reduced-horizon proxy protocol fixes the pre-training recipe. On a 449-task EDA-scripting benchmark spanning three languages and four commercial runtimes, we ask whether it matches deployed Flash masters at comparable accuracy while materially reducing per-subtask cost, and what an on-premise subagent must co-locate with the model.
- **Multimodal grounding for visual EDA artifacts.** Layout, schematic, and waveform views carry much of the correctness signal in custom and analog design, yet a text-only subagent cannot observe them. BaiZe pairs its language core with a from-scratch vision encoder and tests whether a 196-token-per-image budget suffices for such detail-dense views. The encoder reaches 68.2% linear-probe on an internal protocol, yet the MLLM stage averages 26.1% over 11 benchmarks: encoder-quality gains do not transfer downstream at the 224-px single-tile setting.

2 Background and Related Work

2.1 Small Language Models and Agentic Reasoning

Parameter-efficient SLMs (< 2 B) have recently narrowed the gap with larger models on mathematical and commonsense reasoning. MiniCPM **?** introduced scalable training strategies and domain-weighted data mixing for small models; Qwen **?** and Gemma **?** demonstrate that strong code- and math-enriched corpora can lift small models substantially. Most of these efforts, however, treat the final trained model as the deliverable and report comparatively little about the *search* that produced it. Complementarily, agentic instruction-tuning datasets (e.g., tool-use and multi-step reasoning corpora) **?** have been shown to elicit planning and tool-invocation behaviour, yet are rarely paired with a from-scratch pre-training recipe for the 1B regime.

2.2 Hybrid and State-Space Architectures

State-space models (SSMs) such as Mamba **?** and Mamba-2 **?** replace full attention with a recurrent scan update, offering linear-time decoding that suits inference-bound deployments. Hybrid designs interleave a small number of attention layers among SSM/MLP layers—notably Nemotron-H **?** and Hymba **?**—to recover long-range context while retaining fast decode. Released Megatron-Core variants of these families are scarce below 2B, leaving their small-scale trade-offs under-documented; our controlled 2B comparison (Section **??**) addresses precisely this gap.

2.3 Multimodal Vision-Language Models

Vision-language models connect a vision encoder to a language model through a lightweight projector, enabling end-to-end visual understanding and instruction following. LLaVA **?** established visual instruction tuning—aligning a vision encoder to a frozen LLM via a linear projection followed by instruction fine-tuning—as a scalable recipe for open multimodal assistants. LLaVA-OneVision **?** consolidates this line into a single open model spanning single-image, multi-image, and video tasks through a staged curriculum: stage-1 projector pre-training, stage-1.5 mid-training, and stage-2 instruction tuning. BaiZe follows this curriculum in stage (iv) (Section **??**), substituting a from-scratch vision encoder (stage iii) for the frozen off-the-shelf encoder.

2.4 LLMs for Electronic Design Automation

A growing line of work applies LLM agents to chip design tasks such as register-transfer-level generation, verification, and bug triage **??**. These applications motivate a cheap, always-on, agentic multimodal core that can run per-team or per-engineer; our work supplies the efficient multimodal backbone such systems require.

Table 1: BaiZe language-backbone configuration (Mamba2-hybrid).

Hyper-parameter	Value
Transformer layers	56
Architecture	hybrid (Mamba-2 + GQA + MLP)
Mamba-2 / MLP layers	24 / 28
Attention layers	4 (interleaved)
Hidden size	2048
FFN hidden size	8192
Attention heads (Q)	16
Query groups (GQA)	4
Mamba-2 state-space groups	8
Vocabulary size	129,281
Sequence length	16,384
Activation	SiLU / SwiGLU
Normalisation	RMSNorm
Precision	FP8 tensorwise
Parameters	≈2.22B

3 Architecture and Pipeline Overview

BaiZe is an end-to-end multimodal agentic system assembled in five stages: (i) from-scratch pre-training of a hybrid language backbone, (ii) post-training of that backbone (instruction tuning and reinforcement learning to strengthen agentic capability), (iii) pre-training of a custom vision encoder, (iv) alignment of the two into a multimodal large language model, and (v) semiconductor-domain adaptation via reinforcement learning. This section fixes the shared scaffolding—the two backbones, the overall pipeline, and the global training environment—before each stage is detailed in its own section.

3.1 Language Backbone

The language backbone is a sequential hybrid model **??**: 56 transformer layers interleave Mamba-2 state-space layers, four grouped-query-attention (GQA) layers, and MLP layers. The layer allocation follows the Nemotron-H sequential hybrid convention—four attention layers among the 56, the remainder Mamba-2 and MLP—as provided by Megatron-Core's Mamba-hybrid recipe, whose 56-layer pattern we reuse verbatim while halving only the hidden and FFN widths to reach the ≈2B scale. BaiZe is not a Nemotron-H model: it is an order of magnitude smaller, contains no mixture-of-experts layers, and is pre-trained from scratch on an independent corpus under a fixed GPU-hour budget. Each Mamba-2 layer uses 8 state-space groups; the four attention layers use 16 query heads and 4 query groups (GQA) with untied word embeddings. Crucially, only the four attention layers carry rotary position embeddings (RoPE); the 52 non-attention layers (24 Mamba-2 SSM and 28 MLP) are position-agnostic, so the backbone's long-context extrapolation quality depends on just $4/56{=}7.1\%$ of layers, an asymmetry exploited in Section **??**. The full configuration is listed in Table **??**; the model totals ≈2.22B parameters.

3.2 Vision Encoder

The vision encoder is pre-trained from scratch in stage (iii) and is selected from iso-parameter candidates at the 500–600 M scale; Section **??** reports the comparison and its outcome.

3.3 Pipeline and Global Environment

Table **??** summarises the five-stage pipeline. Stage (i) production training runs on two $8\times$NVIDIA H100 nodes (16 GPUs; CUDA 12.8, PyTorch 2.8.0, Python 3.10); other stages use a single $8\times$H100 node. Fused kernels, selective activation recomputation, and cross-entropy loss fusion are enabled where applicable, with `tensor_model_parallel_size` $= 1$; Stage (i) trains in FP8 tensorwise at the main sequence length, reverting to BF16 at shorter lengths.

Table 2: BaiZe five-stage pipeline overview.

Stage	Task	Status	Section
(i)	LM pre-training	in progress	**??**
(ii)	Post-training (SFT, RLVR)	design	**??**
(iii)	Vision encoder	complete	**??**
(iv)	MLLM alignment	in progress	**??**
(v)	Domain adapt. (RLHF)	design	**??**

4 Vision Encoder Pretraining

Stage (iii) pre-trains a custom vision encoder from scratch (Section **??**); we summarise the outcome here and defer the design study to a companion report.

Selected encoder. We adopt **OpenVision2**, a pure-attention ViT at width-512/depth-30 (126.8M parameters), trained from scratch with a bidirectional trunk at 224px/16-px patches on a ≈18.5M-pair image–text corpus (CC12M + Amshaker). The objective combines a global InfoNCE contrastive term with a caption-*independent* masked-patch reconstruction term at a 1:1 weight and a 0.6 mask ratio.

Final result. On the frozen trunk with an ImageNet-1k linear probe, the encoder reaches $11.50\% \rightarrow \mathbf{19.76}\%$ top-1 from 5.1M to 55.3M samples (CC12M+Amshaker, 18.5M unique pairs), and 20.27% at 176M samples on a 58.8M-pair corpus (adding GPIC), with no saturation. This is +12.4 pp above a contrastive-only baseline at the same sample budget, and it is the operating point used by stage (iv). The $224/16$ resolution is retained because it is best on both loss and throughput; whether 196 visual tokens suffice for detail-dense layout and schematic views is tested in stage (iv) (Section **??**).

Encoder upgrade. A second tower—ViT-L/14 (304M) trained with open_clip under the same objective but lower peak LR (5×10^{-4}) and longer warm-up (3000 steps)—reaches $\mathbf{68.236 \pm 0.018}\%$ top-1 (5-NN 58.904%) on ImageNet-1k (Protocol B, 3 seeds), versus 64.337% / 55.956% for the ViT-B/16 (86M) tower; the +3.9 pp gap is a joint effect of tower size, LR, and warm-up, not a capacity-only attribution.

5 MLLM Alignment

Stage (iv) connects the vision encoder (Section **??**) to the language backbone via a lightweight projector, following the staged curriculum of open multimodal assistants; layout and schematic views account for 70.2% of benchmark tasks **?**, so a multimodal core shares the same execution-based protocol as the text-only stage.

Tasks and curriculum. Three task families each admit objective ground truth: (i) *image→script* (rendered view → PyAether/SKILL script, self-labelled by execution); (ii) *script→image* (predict geometry, self-labelled); (iii) *layout VQA* (attribute questions, expert-labelled). The first family shares the assertion sandbox of stage (ii), giving a common metric. We run stage-1 projector pre-training on captions, stage-1.5 mid-training on the three families, and stage-2 instruction tuning on the full mixture; ablations vary the projector type, tower freezing, and visual-token count. The $224/16$ setting (196 tokens/image) is retained; whether it transfers to detail-dense line drawings is what stage (iv) tests.

Downstream results. Table **??** reports the Stage-2 step-3500 checkpoint on 12 lmms-eval benchmarks for ViT-L/14 (304M) and ViT-B/16 (86M). The 11 accuracy benchmarks average to 26.10% vs. 26.12% (−0.02 pp, indistinguishable); despite the +3.9 pp encoder advantage (Section **??**), downstream metrics are flat, suggesting the bottleneck lies not in tower capacity but in the 224-px single-tile setting, the MLLM architecture, or weak proxy-to-downstream transfer.

Table 3: MLLM downstream results (Stage-2 step 3500, %; internal lmms-eval, single seed, 224-px single-tile). MME† is excluded from the 11-set mean.

Benchmark	ViT-L	ViT-B	Benchmark	ViT-L	ViT-B
AI2D	60.65	60.49	CountBench	10.59	9.98
RealWorldQA	45.36	46.41	DocVQA	7.35	7.44
MMMU_val	44.00	42.67	OCRBench	1.60	1.60
BLINK	38.88	39.59	InfoVQA	16.18	16.59
MMStar	27.17	27.07	ChartQA	12.16	12.92
MMMU_Pro	23.12	22.60	**11-set avg**	**26.10**	**26.12**
MME† (perc./cog./total): 513.75/204.29/718.04				559.67/246.79/806.45	

6 Semiconductor Domain Adaptation

Stage (v) completes the pipeline by adapting the multimodal model to proprietary design practice. It differs from stage (ii) in two respects, which we state explicitly because otherwise the two stages would coincide: the reward is no longer verifiable but *learned from human judgement*, and the target behaviour is no longer general API usage but a team's internal conventions—naming rules, layer policies, review checklists—which no public benchmark measures.

Method. We collect pairwise judgements from in-house design reviews over the stage-(iv) task families, fit a domain reward model on them, and optimise the aligned model against that model under a KL-regularised objective. Because preference data is far scarcer than the verifiable signal of stage (ii), we initialise from the stage-(iv) checkpoint and constrain the update rather than training the objective from the pre-trained model.

Why a small model. The rationale for a $\approx$2.2B core is strongest here, and it is a constraint rather than a preference. Proprietary layouts and proprietary preference data cannot leave the premises, so the choice is not between a large hosted model and a small local one—it is between a small local model and no model at all. Serving cost then dominates, and for an agentic session the relevant cost is not throughput on a short prompt but memory that grows with accumulated context: in the hybrid backbone of Section **??**, the four attention layers are the only component whose serving state grows with the session, while the Mamba-2 layers hold a constant-size recurrent state.

Evaluation. We evaluate on held-out review items using expert agreement, and report the result beside the execution-based numbers of stage (ii) so that verifiable and preference-based adaptation can be compared on the same system rather than in separate accounts.

Results are reported as **[TBD]**.

7 Conclusion

We presented BaiZe, an end-to-end multimodal agentic system: a $\approx$2.2B-parameter hybrid (state-space + attention) language model built from scratch, post-trained for agentic capability, paired with a custom vision encoder, aligned into a multimodal model, and adapted to the semiconductor domain. Through a controlled from-scratch comparison, we selected a sequential Mamba2-hybrid backbone over a dense transformer for its lower loss-per-parameter and substantially faster inference, then applied a staged, budget-constrained hyper-parameter search (S1–S5) on the final backbone. Stages (ii) to (v) are specified here as a protocol and evaluated on a common execution-based benchmark **?**; their measurements are reported as **[TBD]**. The complete recipe and a single reproducible command encapsulate the entire construction under the NVIDIA NeMo recipe.

The immediate next step is agentic fine-tuning with the `UltraData-SFT-Agent` corpus toward a deployable agent core for EDA and embedded applications. We hope this blueprint lowers the barrier for cost-bounded teams building their own efficient agentic SLMs from scratch.